3

Data Engineering

This chapter will begin exploring the LLM Twin project in more depth. We will learn how to design and implement the data collection pipeline to gather the raw data we will use in all our LLM use cases, such as fine-tuning or inference. As this is not a book on data engineering, we will keep this chapter short and focus only on what is strictly necessary to collect the required raw data. Starting with *Chapter 4*, we will concentrate on LLMs and GenAI, exploring its theory and concrete implementation details.

When working on toy projects or doing research, you usually have a static dataset with which you work. But in our LLM Twin use case, we want to mimic a real-world scenario where we must gather and curate the data ourselves. Thus, implementing our data pipeline will connect the dots regarding how an end-to-end ML project works. This chapter will explore how to design and implement an **Extract, Transform, Load** (**ETL**) pipeline that crawls multiple social platforms, such as Medium, Substack, or GitHub, and aggregates the gathered data into a MongoDB data warehouse. We will show you how to implement various crawling methods, standardize the data, and load it into a data warehouse.

We will begin by designing the LLM Twin's data collection pipeline and explaining the architecture of the ETL pipeline. Afterward, we will move directly to implementing the pipeline, starting with ZenML, which will

orchestrate the entire process. We will investigate the crawler implementation and understand how to implement a dispatcher layer that instantiates the right crawler class based on the domain of the provided link while following software best practices. Next, we will learn how to implement each crawler individually. Also, we will show you how to implement a data layer on top of MongoDB to structure all our documents and interact with the database.

Finally, we will explore how to run the data collection pipeline using ZenML and query the collected data from MongoDB.

Thus, in this chapter, we will study the following topics:

- Designing the LLM Twin's data collection pipeline
- Implementing the LLM Twin's data collection pipeline
- Gathering raw data into the data warehouse

By the end of this chapter, you will know how to design and implement an ETL pipeline to extract, transform, and load raw data ready to be ingested into the ML application.

Designing the LLM Twin’s data collection pipeline

Before digging into the implementation, we must understand the LLM Twin’s data collection ETL architecture, illustrated in *Figure 3.1*. We must explore what platforms we will crawl to extract data from and how we will design our data structures and processes. However, the first step is understanding how our data collection pipeline maps to an ETL process.

An ETL pipeline involves three fundamental steps:

1. We **extract** data from various sources. We will crawl data from platforms like Medium, Substack, and GitHub to gather raw data.
2. We **transform** this data by cleaning and standardizing it into a consistent format suitable for storage and analysis.
3. We **load** the transformed data into a data warehouse or database.

For our project, we use MongoDB as our NoSQL data warehouse. Although this is not a standard approach, we will explain the reasoning behind this choice shortly.

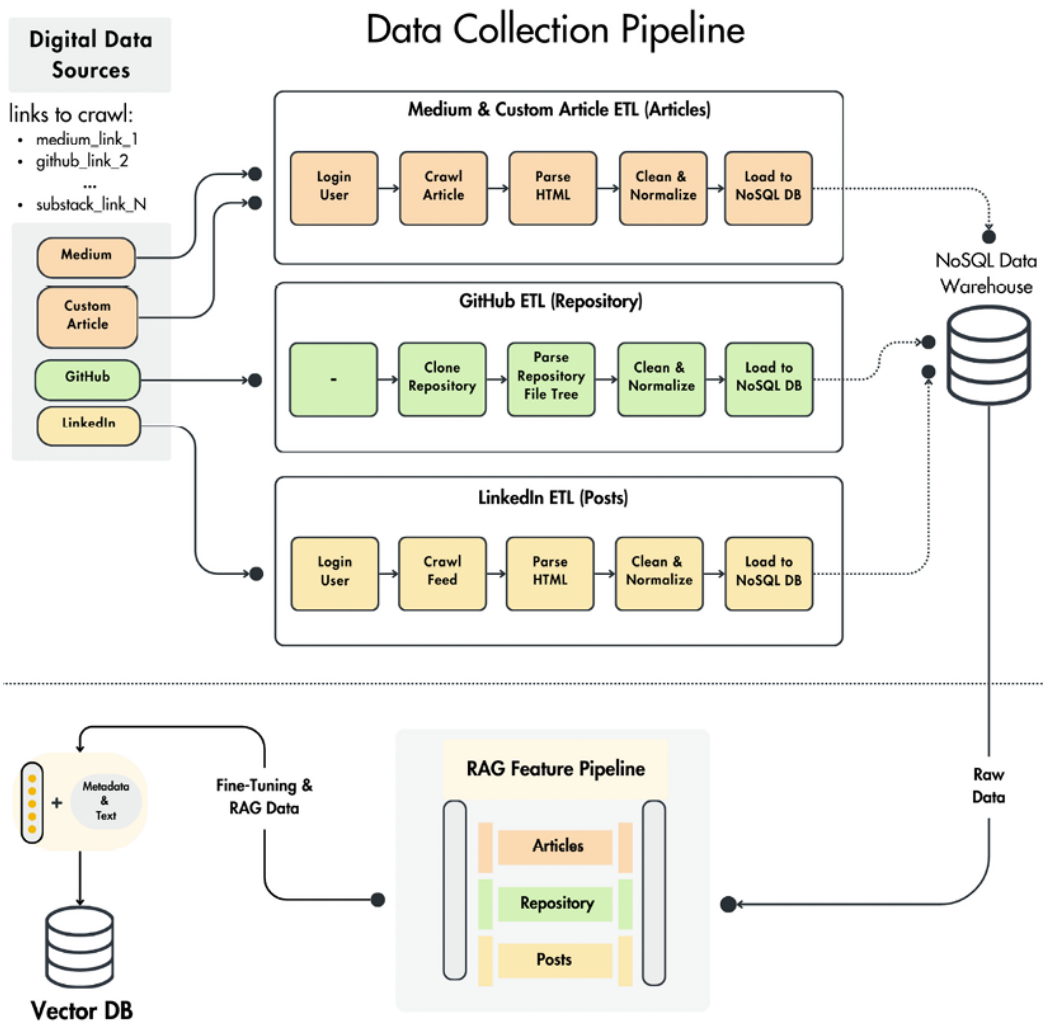


Figure 3.1: LLM Twin's data collection ETL pipeline architecture

We want to design an ETL pipeline that inputs a user and a list of links as input. Afterward, it crawls each link individually, standardizes the collected content, and saves it under that specific author in a MongoDB data warehouse.

Hence, the signature of the data collection pipeline will look as follows:

- **Input:** A list of links and their associated user (the author)
- **Output:** A list of raw documents stored in the NoSQL data warehouse

We will use `user` and `author` interchangeably, as in most scenarios across the ETL pipeline, a user is the author of the extracted content. However, within the data warehouse, we have only a user collection.

The ETL pipeline will detect the domain of each link, based on which it will call a specialized crawler. We implemented four different crawlers for three different data categories, as seen in *Figure 3.2*. First, we will explore the three fundamental data categories we will work with across the book. All our collected documents can be boiled down to an article, repository (or code), and post. It doesn't matter where the data comes from. We are primarily interested in the document's format. In most scenarios, we will have to process these data categories differently. Thus, we created a different domain entity for each, where each entity will have its class and collection in MongoDB. As we save the source URL within the document's metadata, we will still know its source and can reference it in our GenAI use cases.

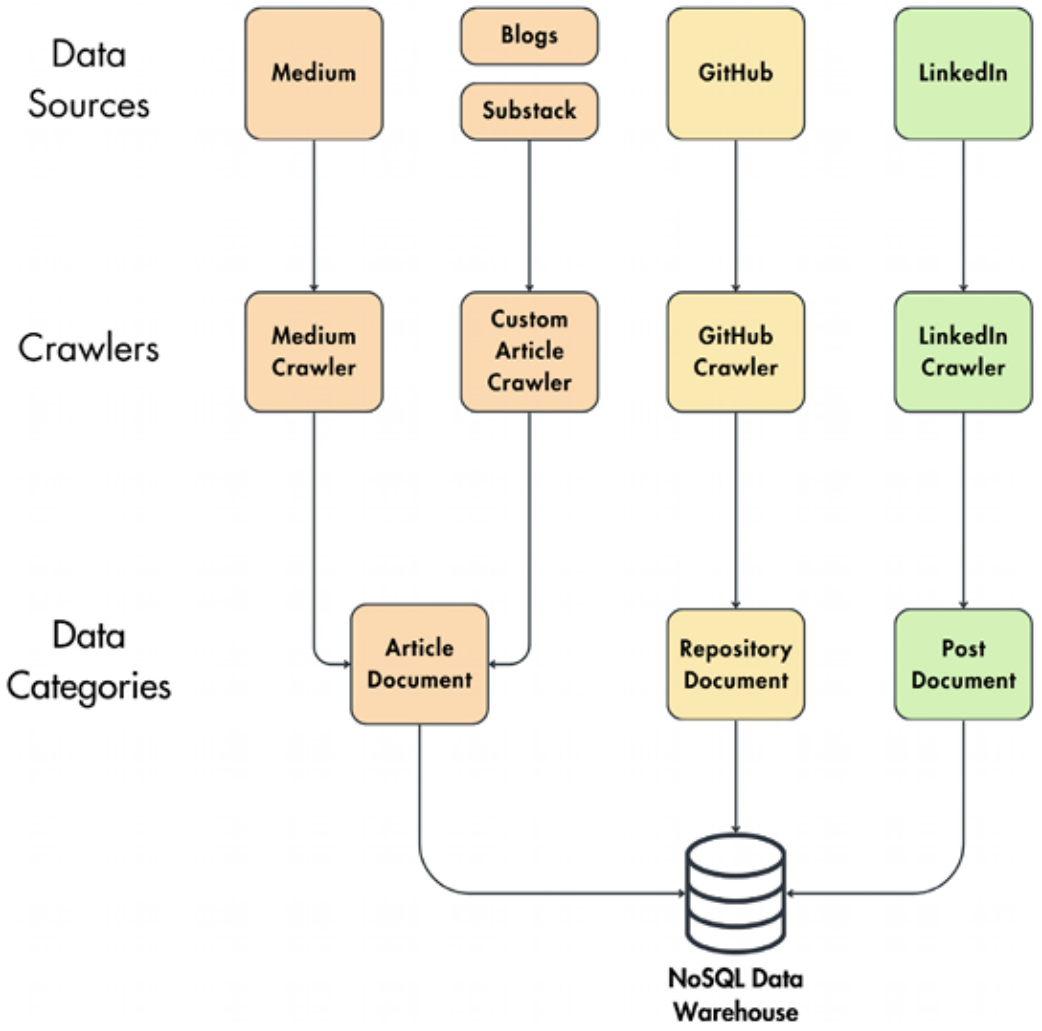


Figure 3.2: The relationship between the crawlers and the data categories

Our codebase supports four different crawlers:

- **Medium crawler**: Used to collect data from Medium. It outputs an article document. It logs in to Medium and crawls the HTML of the article's link. Then, it extracts, cleans, and normalizes the text from the HTML and loads the standardized text of the article into the NoSQL data warehouse.
- **Custom article crawler**: It performs similar steps to the Medium crawler but is a more generic implementation for collecting articles from various sites. Thus, as it doesn't implement any particularities of

any platform, it doesn’t perform the login step and blindly gathers all the HTML from a particular link. This is enough for articles freely available online, which you can find on Substack and people’s blogs. We will use this crawler as a safety net when the link’s domain isn’t associated with the other supported crawlers. For example, when providing a Substack link, it will default to the custom article crawler, but when providing a Medium URL, it will use the Medium crawler.

- **GitHub crawler**: This collects data from GitHub. It outputs a repository document. It clones the repository, parses the repository file tree, cleans and normalizes the files, and loads them to the database.
- **LinkedIn crawler**: This is used to collect data from LinkedIn. It outputs multiple post documents. It logs in to LinkedIn, navigates to the user’s feed, and crawls all the user’s latest posts. For each post, it extracts its HTML, cleans and normalizes it, and loads it to MongoDB.

In the next section, we will examine each crawler’s implementation in detail. For now, note that each crawler accesses a specific platform or site in a particular way and extracts HTML from it. Afterward, all the crawlers parse the HTML, extract the text from it, and clean and normalize it so it can be stored in the data warehouse under the same interface.

By reducing all the collected data to three data categories and not creating a new data category for every new data source, we can easily extend this architecture to multiple data sources with minimal effort. For example, if we want to start collecting data from X, we only have to implement a new crawler that outputs a post document, and that’s it. The rest of the code will remain untouched. Otherwise, if we introduced the source dimension in the class and document structure, we would have to add code to all downstream layers to support any new data source. For example, we would have to

implement a new document class for each new source and adapt the feature pipeline to support it.

For our proof of concept, crawling a few hundred documents is enough, but if we want to scale it to a real-world product, we would probably need more data sources to crawl from. LLMs are data-hungry. Thus, you need thousands of documents for ideal results instead of just a few hundred. But in many projects, it's an excellent strategy to implement an end-to-end project version that isn't the most accurate and iterate through it later. Thus, by using this architecture, you can easily add more data sources in future iterations to gather a larger dataset. More on LLM fine-tuning and dataset size will be covered in the next chapter.

How is the ETL process connected to the feature pipeline? The feature pipeline ingests the raw data from the MongoDB data warehouse, cleans it further, processes it into features, and stores it in the Qdrant vector DB to make it accessible for the LLM training and inference pipelines. *Chapter 4* provides more information on the feature pipeline. The ETL process is independent of the feature pipeline. The two pipelines communicate with each other strictly through the MongoDB data warehouse. Thus, the data collection pipeline can write data for MongoDB, and the feature pipeline can read from it independently and on different schedules.

Why did we use MongoDB as a data warehouse? Using a transactional database, such as MongoDB, as a data warehouse is uncommon. However, in our use case, we are working with small amounts of data, which MongoDB can handle. Even if we plan to compute statistics on top of our MongoDB collections, it will work fine at the scale of our LLM Twin's data (hundreds of documents). We picked MongoDB to store our raw data primarily because of the nature of our unstructured data: text crawled from

the internet. By mainly working with unstructured text, selecting a NoSQL database that doesn’t enforce a schema made our development easier and faster. Also, MongoDB is stable and easy to use. Their Python SDK is intuitive. They provide a Docker image that works out of the box locally and a cloud freemium tier that is perfect for proofs of concept, such as the LLM Twin. Thus, we can freely work with it locally and in the cloud. However, when working with big data (millions of documents or more), using a dedicated data warehouse such as Snowflake or BigQuery will be ideal.

Now that we’ve understood the architecture of the LLM Twin’s data collection pipeline, let’s move on to its implementation.

Implementing the LLM Twin’s data collection pipeline

As we presented in *Chapter 2*, the entry point to each pipeline from our LLM Twin project is a ZenML pipeline, which can be configured at runtime through YAML files and run through the ZenML ecosystem. Thus, let’s start by looking into the ZenML `digital_data_etl` pipeline. You’ll notice that this is the same pipeline we used as an example in *Chapter 2* to illustrate ZenML. But this time, we will dig deeper into the implementation, explaining how the data collection works behind the scenes. After understanding how the pipeline works, we will explore the implementation of each crawler used to collect data from various sites and the MongoDB documents used to store and query data from the data warehouse.

ZenML pipeline and steps

In the code snippet below, we can see the implementation of the ZenML `digital_data_etl` pipeline, which inputs the user's full name and a list of links that will be crawled under that user (considered the author of the content extracted from those links). Within the function, we call two steps. In the first one, we look up the user in the database based on its full name. Then, we loop through all the links and crawl each independently. The pipeline's implementation is available in our repository at `pipelines/digital_data_etl.py`.

```
from zenml import pipeline
from steps.etl import crawl_links, get_or_create_user
@pipeline
def digital_data_etl(user_full_name: str, links: list[str]) ->
    user = get_or_create_user(user_full_name)
    last_step = crawl_links(user=user, links=links)
    return last_step.invocation_id
```

Figure 3.3 shows a run of the `digital_data_etl` pipeline on the ZenML dashboard. The next phase is to explore the `get_or_create_user` and `crawl_links` ZenML steps individually. The step implementation is available in our repository at `steps/etl`.

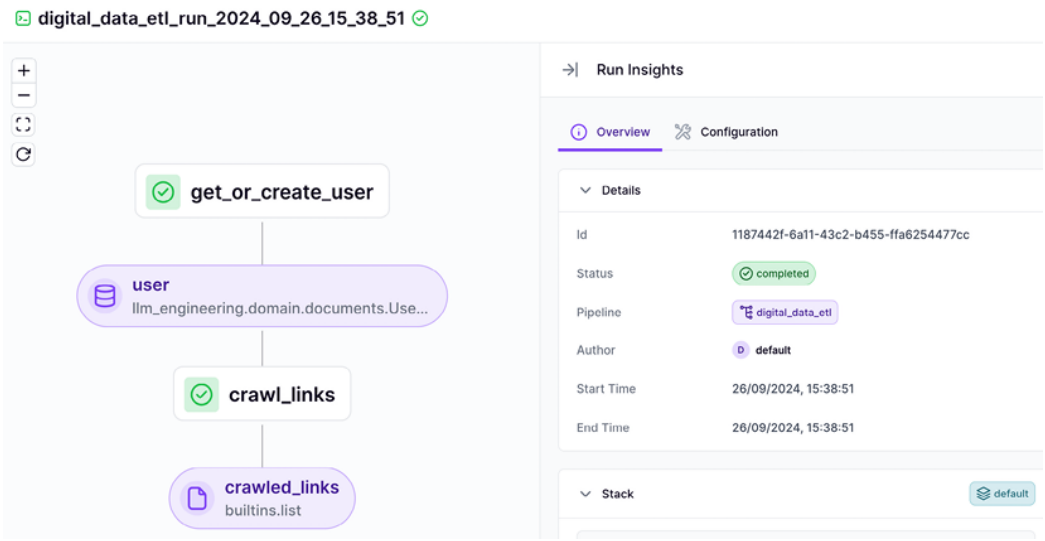


Figure 3.3: Example of a digital_data_etl pipeline run from ZenML's dashboard

We will start with the `get_or_create_user` ZenML step. We begin by importing the necessary modules and functions used throughout the script.

```
from loguru import logger
from typing_extensions import Annotated
from zenml import get_step_context, step
from llm_engineering.application import utils
from llm_engineering.domain.documents import UserDocument
```

Next, we define the function's signature, which takes a user's full name as input and retrieves an existing user or creates a new one in the MongoDB database if it doesn't exist:

```
@step
def get_or_create_user(user_full_name: str) -> Annotated[UserDo
```

Using a utility function, we split the full name into first and last names. Then, we attempt to retrieve the user from the database or create a new one if it doesn't exist. We also retrieve the current step context and add

metadata about the user to the output, which will be reflected in the metadata of the `user` ZenML output artifact:

```
    logger.info(f"Getting or creating user: {user_full_name}")
    first_name, last_name = utils.split_user_full_name(user_ful
    user = UserDocument.get_or_create(first_name=first_name, la
    step_context = get_step_context()
    step_context.add_output_metadata(output_name="user", metada
    return user
```

Additionally, we define a helper function called `_get_metadata()`, which builds a dictionary containing the query parameters and the retrieved user information, which will be added as metadata to the user artifact:

```
def _get_metadata(user_full_name: str, user: UserDocument) -> d
    return {
        "query": {
            "user_full_name": user_full_name,
        },
        "retrieved": {
            "user_id": str(user.id),
            "first_name": user.first_name,
            "last_name": user.last_name,
        },
    }
```

We will move on to the `crawl_links` ZenML step, which collects the data from the provided links. The code begins by importing essential modules and libraries for web crawling:

```
from urllib.parse import urlparse
from loguru import logger
from tqdm import tqdm
from typing_extensions import Annotated
```

```
from zenml import get_step_context, step
from llm_engineering.application.crawlers.dispatcher import Cra
from llm_engineering.domain.documents import UserDocument
```

Following the imports, the main function inputs a list of links written by a specific author. Within this function, a crawler dispatcher is initialized and configured to handle specific domains such as LinkedIn, Medium, and GitHub:

```
@step
def crawl_links(user: UserDocument, links: list[str]) -> Annota
    dispatcher = CrawlerDispatcher.build().register_linkedin().
    logger.info(f"Starting to crawl {len(links)} link(s).")
```

The function initializes variables to store the output metadata and count successful crawls. It then iterates over each link. It attempts to crawl and extract data for each link, updating the count of successful crawls and accumulating metadata about each URL:

```
    metadata = {}
    successfull_crawls = 0
    for link in tqdm(links):
        successfull_crawl, crawled_domain = _crawl_link(dispatc
        successfull_crawls += successfull_crawl
        metadata = _add_to_metadata(metadata, crawled_domain, s
```

After processing all links, the function attaches the accumulated metadata to the output artifact:

```
    step_context = get_step_context()
    step_context.add_output_metadata(output_name="crawled_links
    logger.info(f"Successfully crawled {successfull_crawls} / {
```

```
links.")
    return links
```

The code includes a helper function that attempts to extract information from each link using the appropriate crawler based on the link's domain. It handles any exceptions that may occur during extraction and returns a tuple indicating the crawl's success and the link's domain:

```
def _crawl_link(dispatcher: CrawlerDispatcher, link: str, user:
    crawler = dispatcher.get_crawler(link)
    crawler_domain = urlparse(link).netloc
    try:
        crawler.extract(link=link, user=user)
        return (True, crawler_domain)
    except Exception as e:
        logger.error(f"An error occurred while crawling: {e!s}"
        return (False, crawler_domain)
```

Another helper function is provided to update the metadata dictionary with the results of each crawl:

```
def _add_to_metadata(metadata: dict, domain: str, successfull_c
    if domain not in metadata:
        metadata[domain] = {}
    metadata[domain]["successful"] = metadata.get(domain, {}).g
    metadata[domain]["total"] = metadata.get(domain, {}).get("t
    return metadata
```

As seen in the abovementioned `_crawl_link()` function, the `CrawlerDispatcher` class knows what crawler to initialize based on each link's domain. The logic is then abstracted away under the crawler's

`extract()` method. Let’s zoom in on the `CrawlerDispatcher` class to understand how this works fully.

The dispatcher: How do you instantiate the right crawler?

The entry point to our crawling logic is the `CrawlerDispatcher` class. As illustrated in *Figure 3.4*, the dispatcher acts as the intermediate layer between the provided links and the crawlers. It knows what crawler to associate with each URL.

The `CrawlerDispatcher` class knows how to extract the domain of each link and initialize the proper crawler that collects the data from that site. For example, if it detects the https://medium.com domain when providing a link to an article, it will build an instance of the `MediumCrawler` used to crawl that particular platform. With that in mind, let's explore the implementation of the `CrawlerDispatcher` class.



All the crawling logic is available in the GitHub repository at `llm_engineering/application/crawlers`.

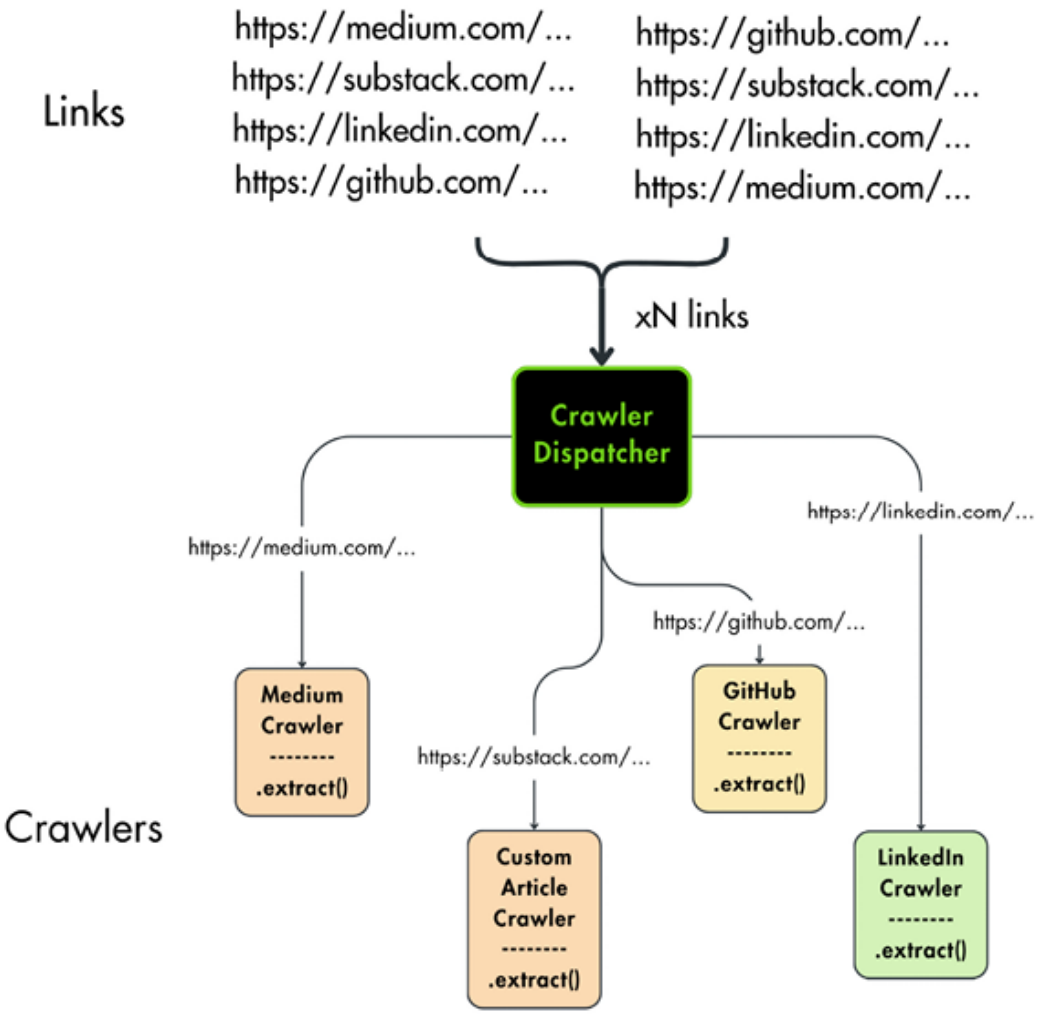


Figure 3.4: The relationship between the provided links, the CrawlerDispatcher, and the crawlers

We begin by importing the necessary Python modules for URL handling and regex, along with importing our crawler classes:

```
import re
from urllib.parse import urlparse
from loguru import logger
from .base import BaseCrawler
from .custom_article import CustomArticleCrawler
from .github import GithubCrawler
from .linkedin import LinkedInCrawler
from .medium import MediumCrawler
```

The `CrawlerDispatcher` class is defined to manage and dispatch appropriate crawler instances based on given URLs and their domains. Its constructor initializes a registry to store the registered crawlers.

```
class CrawlerDispatcher:
    def __init__(self) -> None:
        self._crawlers = {}
```

As we are using the builder creational pattern to instantiate and configure the dispatcher, we define a `build()` class method that returns an instance of the dispatcher:

```
    @classmethod
    def build(cls) -> "CrawlerDispatcher":
        dispatcher = cls()
        return dispatcher
```

The dispatcher includes methods to register crawlers for specific platforms like Medium, LinkedIn, and GitHub. These methods use a generic `register()` method under the hood to add each crawler to the registry. By returning self, we follow the builder creational pattern (more on the builder pattern: https://refactoring.guru/design-patterns/builder). We can chain multiple `register_*()` methods when instantiating the dispatcher as follows: `CrawlerDispatcher.build().register_linkedin().register_medium()`.

```
    def register_medium(self) -> "CrawlerDispatcher":
        self.register("https://medium.com", MediumCrawler)
        return self
    def register_linkedin(self) -> "CrawlerDispatcher":
        self.register("https://linkedin.com", LinkedInCrawler)
        return self
```

```
    def register_github(self) -> "CrawlerDispatcher":
        self.register("https://github.com", GithubCrawler)
        return self
```

The generic `register()` method normalizes each domain to ensure its format is consistent before it's added as a key to the `self._crawlers` registry of the dispatcher. This is a critical step, as we will use the key of the dictionary as the domain pattern to match future links with a crawler:

```
    def register(self, domain: str, crawler: type[BaseCrawler])
        parsed_domain = urlparse(domain)
        domain = parsed_domain.netloc
        self._crawlers[r"https://(www\.)?{}/*".format(re.escape
```

Finally, the `get_crawler()` method determines the appropriate crawler for a given URL by matching it against the registered domains. If no match is found, it logs a warning and defaults to using the `CustomArticleCrawler`.

```
    def get_crawler(self, url: str) -> BaseCrawler:
        for pattern, crawler in self._crawlers.items():
            if re.match(pattern, url):
                return crawler()
        else:
            logger.warning(f"No crawler found for {url}. Defaul
            return CustomArticleCrawler()
```

The next step in understanding how the data collection pipeline works is analyzing each crawler individually.

The crawlers

Before exploring each crawler's implementation, we must present their base class, which defines a unified interface for all the crawlers. As shown in *Figure 3.4*, we can implement the dispatcher layer because each crawler follows the same signature. Each class implements the `extract()` method, allowing us to leverage OOP techniques such as polymorphism, where we can work with abstract objects without knowing their concrete subclass. For example, in the `_crawl_link()` function from the ZenML steps, we had the following code:

```
crawler = dispatcher.get_crawler(link)
crawler.extract(link=link, user=user)
```

Note how we called the `extract()` method without caring about what specific type of crawler we instantiated. To conclude, working with abstract interfaces ensures core reusability and ease of extension.

Base classes

Now, let's explore the `BaseCrawler` interface, which can be found in the repository at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/crawlers/base.py.

```
from abc import ABC, abstractmethod
class BaseCrawler(ABC):
    model: type[NoSQLBaseDocument]
```

```
    @abstractmethod
    def extract(self, link: str, **kwargs) -> None: ...
```

As mentioned above, the interface defines an `extract()` method that takes as input a link. Also, it defines a model attribute at the class level that represents the data category document type used to save the extracted data into the MongoDB data warehouse. Doing so allows us to customize each subclass with different data categories while preserving the same attributes at the class level. We will soon explore the `NoSQLBaseDocument` class when digging into the document entities.

We also extend the `BaseCrawler` class with a `BaseSeleniumCrawler` class, which implements reusable functionality that uses Selenium to crawl various sites, such as Medium or LinkedIn. **Selenium** is a tool for automating web browsers. It's used to interact with web pages programmatically (like logging into LinkedIn, navigating through profiles, etc.).

Selenium can programmatically control various browsers such as Chrome, Firefox, or Brave. For these specific platforms, we need Selenium to manipulate the browser programmatically to log in and scroll through the newsfeed or article before being able to extract the entire HTML. For other sites, where we don't have to go through the login step or can directly load the whole page, we can extract the HTML from a particular URL using more straightforward methods than Selenium.



For the Selenium-based crawlers to work, you must install Chrome on your machine (or a Chromium-based browser such as Brave).

The code begins by setting up the necessary imports and configurations for web crawling using Selenium and the ChromeDriver initializer. The `chromedriver_autoinstaller` ensures that the appropriate version of ChromeDriver is installed and added to the system path, maintaining compatibility with the installed version of your Google Chrome browser (or other Chromium-based browser). Selenium will use the ChromeDriver to communicate with the browser and open a headless session, where we can programmatically manipulate the browser to access various URLs, click on specific elements, such as buttons, or scroll through the newsfeed. Using the `chromedriver_autoinstaller`, we ensure we always have the correct ChromeDriver version installed that matches our machine's Chrome browser version.

```
import time
from tempfile import mkdtemp
import chromedriver_autoinstaller
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from llm_engineering.domain.documents import NoSQLBaseDocument
# Check if the current version of chromedriver exists
# and if it doesn't exist, download it automatically,
# then add chromedriver to path
chromedriver_autoinstaller.install()
```

Next, we define the `BaseSeleniumCrawler` class for use cases where we need Selenium to collect the data, such as collecting data from Medium or LinkedIn.

Its constructor initializes various Chrome options to optimize performance, enhance security, and ensure a headless browsing environment. These options disable unnecessary features like GPU rendering, extensions, and

notifications, which can interfere with automated browsing. These are standard configurations when crawling in headless mode:

```
class BaseSeleniumCrawler(BaseCrawler, ABC):
    def __init__(self, scroll_limit: int = 5) -> None:
        options = webdriver.ChromeOptions()

        options.add_argument("--no-sandbox")
        options.add_argument("--headless=new")
        options.add_argument("--disable-dev-shm-usage")
        options.add_argument("--log-level=3")
        options.add_argument("--disable-popup-blocking")
        options.add_argument("--disable-notifications")
        options.add_argument("--disable-extensions")
        options.add_argument("--disable-background-networking")
        options.add_argument("--ignore-certificate-errors")
        options.add_argument(f"--user-data-dir={mkdtemp()}")
        options.add_argument(f"--data-path={mkdtemp()}")
        options.add_argument(f"--disk-cache-dir={mkdtemp()}")
        options.add_argument("--remote-debugging-port=9226")
```

After configuring the Chrome options, the code allows subclasses to set any additional driver options by calling the `set_extra_driver_options()` method. It then initializes the scroll limit and creates a new instance of the Chrome driver with the specified options:

```
        self.set_extra_driver_options(options)
        self.scroll_limit = scroll_limit
        self.driver = webdriver.Chrome(
            options=options,
        )
```

The `BaseSeleniumCrawler` class includes placeholder methods for `set_extra_driver_options()` and `login()`, which subclasses can override

to provide specific functionality. This ensures modularity, as every platform has a different login page with a different HTML structure:

```
    def set_extra_driver_options(self, options: Options) -> Non
        pass
    def login(self) -> None:
        pass
```

Finally, the `scroll_page()` method implements a scrolling mechanism to navigate through pages, such as LinkedIn, up to a specified scroll limit. It scrolls to the bottom of the page, waits for new content to load, and repeats the process until it reaches the end of the page or the scroll limit is exceeded. This method is essential for feeds where the content appears as the user scrolls:

```
    def scroll_page(self) -> None:
        """Scroll through the LinkedIn page based on the scroll
        current_scroll = 0
        last_height = self.driver.execute_script("return docume
        while True:
            self.driver.execute_script("window.scrollTo(0, docu
            time.sleep(5)
            new_height = self.driver.execute_script("return doc
            if new_height == last_height or (self.scroll_limit
                break
            last_height = new_height
            current_scroll += 1
```

We've understood what the base classes of our crawlers look like. Next, we will look into the implementation of the following specific crawlers:

- `GitHubCrawler(BaseCrawler)`
- `CustomArticleCrawler(BaseCrawler)`

- `MediumCrawler(BaseSeleniumCrawler)`



You can find the implementation of the above crawlers in the GitHub repository at https://github.com/PacktPublishing/LLM-Engineers-Handbook/tree/main/llm_engineering/application/crawlers.

GitHubCrawler class

The `GithubCrawler` class is designed to scrape GitHub repositories, extending the functionality of the `BaseCrawler`. We don't have to log in to GitHub through the browser, as we can leverage Git's clone functionality. Thus, we don't have to leverage any Selenium functionality. Upon initialization, it sets up a list of patterns to ignore standard files and directories found in GitHub repositories, such as `.git`, `.toml`, `.lock`, and `.png`, ensuring that unnecessary files are excluded from the scraping process:

```
class GithubCrawler(BaseCrawler):
    model = RepositoryDocument
    def __init__(self, ignore=(".git", ".toml", ".lock", ".png"
        super().__init__()
        self._ignore = ignore
```

Next, we implement the `extract()` method, where the crawler first checks if the repository has already been processed and stored in the database. If it exists, it exits the method to prevent storing duplicates:

```
def extract(self, link: str, **kwargs) -> None:
    old_model = self.model.find(link=link)
    if old_model is not None:
        logger.info(f"Repository already exists in the database
        return
```

If the repository is new, the crawler extracts the repository name from the link. Then, it creates a temporary directory to clone the repository to ensure that the cloned repository is cleaned up from the local disk after it's processed:

```
    logger.info(f"Starting scrapping GitHub repository: {link}"
    repo_name = link.rstrip("/").split("/")[-1]
    local_temp = tempfile.mkdtemp()
```

Within a try block, the crawler changes the current working directory to the `temporary` directory and executes the `git clone` command in a different process:

```
    try:
        os.chdir(local_temp)
        subprocess.run(["git", "clone", link])
```

After successfully cloning the repository, the crawler constructs the path to the cloned repository. It initializes an empty dictionary used to aggregate the content of the files in a standardized way. It walks through the directory tree, skipping over any directories or files that match the ignore patterns. For each relevant file, it reads the content, removes any spaces, and stores it in the dictionary with the file path as the key:

```
        repo_path = os.path.join(local_temp, os.listdir(local_t
        tree = {}
        for root, _, files in os.walk(repo_path):
            dir = root.replace(repo_path, "").lstrip("/")
            if dir.startswith(self._ignore):
                continue
            for file in files:
                if file.endswith(self._ignore):
                    continue
                file_path = os.path.join(dir, file)
                with open(os.path.join(root, file), "r", errors
                    tree[file_path] = f.read().replace(" ", "")
```

It then creates a new instance of the `RepositoryDocument` model, populating it with the repository content, name, link, platform information, and author details. The instance is then saved to MongoDB:

```
        user = kwargs["user"]
        instance = self.model(
            content=tree,
            name=repo_name,
            link=link,
            platform="github",
            author_id=user.id,
            author_full_name=user.full_name,
        )
        instance.save()
```

Finally, whether the scraping succeeds or an exception occurs, the crawler ensures that the temporary directory is removed to clean up any resources used during the process:

```
    except Exception:
        raise
    finally:
```

```
            shutil.rmtree(local_temp)
        logger.info(f"Finished scrapping GitHub repository: {link}"
```

CustomArticleCrawler class

The `CustomArticleCrawler` class takes a different approach to collecting data from the internet. It leverages the `AsyncHtmlLoader` class to read the entire HTML from a link and the `Html2TextTransformer` class to extract the text from that HTML. Both classes are made available by the `langchain_community` Python package, as seen below, where we import all the necessary Python modules:

```
from urllib.parse import urlparse
from langchain_community.document_loaders import AsyncHtmlLoade
from langchain_community.document_transformers.html2text import
from loguru import logger
from llm_engineering.domain.documents import ArticleDocument
from .base import BaseCrawler
```

Next, we define the `CustomArticleCrawler` class, which inherits from `BaseCrawler`. As before, we don't need to log in or use the scrolling functionality provided by Selenium. In the `extract` method, we first check if the article exists in the database to avoid duplicating content:

```
class CustomArticleCrawler(BaseCrawler):
    model = ArticleDocument
    def extract(self, link: str, **kwargs) -> None:
        old_model = self.model.find(link=link)
        if old_model is not None:
            logger.info(f"Article already exists in the databas
            return
```

If the article doesn't exist, we proceed to scrape it. We use the `AsyncHtmlLoader` class to load the HTML from the provided link. After, we transform it into plain text using the `Html2TextTransformer` class, which returns a list of documents. We are only interested in the first document. As we delegate the whole logic to these two classes, we don't control how the content is extracted and parsed. That's why we used this class as a fallback system for domains where we don't have anything custom implemented. These two classes follow the LangChain paradigm, which provides high-level functionality that works decently in most scenarios. It is fast to implement but hard to customize. That is one of the reasons why many developers avoid using LangChain in production use cases:

```
        logger.info(f"Starting scrapping article: {link}")
        loader = AsyncHtmlLoader([link])
        docs = loader.load()
        html2text = Html2TextTransformer()
        docs_transformed = html2text.transform_documents(docs)
        doc_transformed = docs_transformed[0]
```

We get the page content from the extracted document, plus relevant metadata such as the `title`, `subtitle`, `content`, and `language`:

```
        content = {
            "Title": doc_transformed.metadata.get("title"),
            "Subtitle": doc_transformed.metadata.get("descripti
            "Content": doc_transformed.page_content,
            "language": doc_transformed.metadata.get("language"
        }
```

Next, we parse the URL to determine the platform (or domain) from which the article was scraped:

```
        parsed_url = urlparse(link)
        platform = parsed_url.netloc
```

We then create a new instance of the article model, populating it with the extracted content. Finally, we save this instance to the MongoDB data warehouse:

```
        user = kwargs["user"]
        instance = self.model(
            content=content,
            link=link,
            platform=platform,
            author_id=user.id,
            author_full_name=user.full_name,
        )
        instance.save()
        logger.info(f"Finished scrapping custom article: {link}
```

So far, we have seen how to crawl GitHub repositories and random sites using LangChain utility functions. Lastly, we must explore a crawler using Selenium to manipulate the browser programmatically. Thus, we will continue with the `MediumCrawler` implementation.

MediumCrawler class

The code begins by importing essential libraries and defining the `MediumCrawler` class, which inherits from `BaseSeleniumCrawler`:

```
from bs4 import BeautifulSoup
from loguru import logger
from llm_engineering.domain.documents import ArticleDocument
from .base import BaseSeleniumCrawler
class MediumCrawler(BaseSeleniumCrawler):
    model = ArticleDocument
```

Within the `MediumCrawler` class, we leverage the `set_extra_driver_options()` method to extend the default driver options used by Selenium:

```
    def set_extra_driver_options(self, options) -> None:
        options.add_argument(r"--profile-directory=Profile 2")
```

The `extract()` method implements the core functionality, first checking whether the article exists in the database to prevent duplicate entries.

If the article is new, the method proceeds to navigate to the article's link and scroll through the page to ensure all content is loaded:

```
    def extract(self, link: str, **kwargs) -> None:
        old_model = self.model.find(link=link)
        if old_model is not None:
            logger.info(f"Article already exists in the databas
            return
        logger.info(f"Starting scrapping Medium article: {link}
        self.driver.get(link)
        self.scroll_page()
```

After fully loading the page, the method uses `BeautifulSoup` to parse the HTML content and extract the article's title, subtitle, and full text. `BeautifulSoup` is a popular Python library for web scraping and parsing HTML or XML documents. Thus, we used it to extract all the HTML elements we needed from the HTML accessed with Selenium. Finally, we aggregate everything into a dictionary:

```
        soup = BeautifulSoup(self.driver.page_source, "html.par
        title = soup.find_all("h1", class_="pw-post-title")
```

```
        subtitle = soup.find_all("h2", class_="pw-subtitle-para
        data = {
            "Title": title[0].string if title else None,
            "Subtitle": subtitle[0].string if subtitle else Non
            "Content": soup.get_text(),
        }
```

Finally, the method closes the WebDriver to free up resources. It then creates a new `ArticleDocument` instance, populates it with the extracted content and user information provided via `kwargs`, and saves it to the database:

```
        self.driver.close()
        user = kwargs["user"]
        instance = self.model(
            platform="medium",
            content=data,
            link=link,
            author_id=user.id,
            author_full_name=user.full_name,
        )
        instance.save()
        logger.info(f"Successfully scraped and saved article: {
```

With that, we conclude the `MediumCrawler` implementation. The LinkedIn crawler follows a similar pattern to the Medium one, where it uses Selenium to log in and access the feed of a user's latest posts. Then, it extracts the posts and scrolls through the feed to load the next page until a limit is hit. You can check the full implementation in our repository at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/crawlers/linkedin.py.

With the rise of LLMs, collecting data from the internet has become a critical step in many real-world AI applications. Hence, more high-level tools have appeared in the Python ecosystem, such as Scrapy (https://github.com/scrapy/scrapy), which crawls websites and extracts structured data from their pages, and Crawl4AI (https://github.com/unclecode/crawl4ai), which is highly specialized in crawling data for LLMs and AI applications.

In this section, we've looked at implementing three types of crawlers: one that leverages the `git` executable in a subprocess to clone GitHub repositories, one that uses LangChain utilities to extract the HTML of a single web page, and one that leverages Selenium for more complex scenarios where we have to navigate through the login page, scroll the article to load the entire HTML, and extract it into text format. The last step is understanding how the document classes we've used across the chapter, such as the `ArticleDocument`, work.

The NoSQL data warehouse documents

We had to implement three document classes to structure our data categories. These classes define the specific attributes we require for a document, such as the content, author, and source link. It is best practice to structure your data in classes instead of dictionaries, as the attributes we expect for each item are more verbose, reducing run errors. For example, when accessing a value from a Python dictionary, we can never be sure it is present or its type is current. By wrapping our data items with classes, we can ensure each attribute is as expected.

By leveraging Python packages such as Pydantic, we have out-of-the-box type validation, which ensures consistency in our datasets. Thus, we modeled the data categories as the following document classes, which we already used in the code up until point:

- `ArticleDocument` class
- `PostDocument` class
- `RepositoryDocument` class

These are not simple Python data classes or Pydantic models. They support read and write operations on top of the MongoDB data warehouse. To inject the read-and-write functionality into all the document classes without repeating any code, we used the **Object-Document Mapping** (ODM) software pattern, which is based on the **object-relational mapping** (**ORM**) pattern. Thus, let’s first explore ORM, then move to ODM, and, finally, dig into our custom ODM implementation and document classes.

The ORM and ODM software patterns

Before we talk about software patterns, let's see what ORM is. It's a technique that lets you query and manipulate data from a database using an object-oriented paradigm. Instead of writing SQL or API-specific queries, you encapsulate all the complexity under an ORM class that knows how to handle all the database operations, most commonly CRUD operations. Thus, working with ORM removes the need to handle the database operations manually and reduces the need to write boilerplate code manually. An ORM interacts with a SQL database, such as PostgreSQL or MySQL.

Most modern Python applications use ORMs when interacting with the database. Even though SQL is still a popular choice in the data world, you rarely see raw SQL queries in Python backend components. The most popular Python ORM is SQLAlchemy (https://www.sqlalchemy.org/). Also, with the rise of FastAPI, SQLModel is (https://github.com/fastapi/sqlmodel) a common choice, which is a wrapper over SQLAlchemy that makes the integration easier with FastAPI.

For example, using SQLAlchemy, we defined a `User` ORM with the ID and name fields. The `User` ORM is mapped to the `users` table within the SQL database. Thus, when we create a new user and commit it to the database, it is automatically saved to the `users` table. The same applies to all the CRUD operations on top of the `User` class.

```
from sqlalchemy import Column, Integer, String, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
   Base = declarative_base()
# Define a class that maps to the users table.
   class User(Base):
```

```
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
```

Using the `User` ORM, we can quickly insert or query users directly from Python without writing a line of SQL. Note that an ORM usually supports all **CRUD** operations. Here is a code snippet that shows how to save an instance of the User ORM to a SQLite database:

```
engine = create_engine("sqlite:///:memory:")
Base.metadata.create_all(engine)
# Create a session used to interact with the database.
Session = sessionmaker(bind=engine)
session = Session()
# Add a new user.
new_user = User(name="Alice")
session.add(new_user)
session.commit()
```

Also, this is how we can query a user from the `users` SQLite table:

```
user = session.query(User).first()
if user:
print(f"User ID: {user.id}")
print(f"User name: {user.name}")
```



Find the entire script and how to run it in the GitHub repository at `code_snippets/03_orm.py`.

The ODM pattern is extremely similar to ORM, but instead of working with SQL databases and tables, it works with NoSQL databases (such as MongoDB) and unstructured collections. As we work with NoSQL

databases, the data structure is centered on collections, which store JSON-like documents rather than rows in tables.

To conclude, ODM simplifies working with document-based NoSQL databases and maps object-oriented code to JSON-like documents. We will implement a light ODM module on top of MongoDB to fully understand how ODM works.

Implementing the ODM class

This section will explore how to implement an ODM class from scratch. This is an excellent exercise to learn how ODM works and sharpen our skills in writing modular and reusable Python classes. Hence, we will implement a base ODM class called `NoSQLBaseDocument`, from which all the other documents will inherit to interact with the MongoDB data warehouse.



The class can be found in our repository at `llm_engineering/domain/base/nosql.py`.

The code starts by importing essential modules and setting up the database connection. Through the `_database` variable, we establish a connection to the database specified in the settings, which is by default called `twin`:

```
import uuid
from abc import ABC
from typing import Generic, Type, TypeVar
from loguru import logger
from pydantic import UUID4, BaseModel, Field
from pymongo import errors
from llm_engineering.domain.exceptions import ImproperlyConfigu
from llm_engineering.infrastructure.db.mongo import connection
from llm_engineering.settings import settings
_database = connection.get_database(settings.DATABASE_NAME)
```

Next, we define a type variable `T` bound to the `NoSQLBaseDocument` class. The variable leverages Python's generic module, allowing us to generalize the class's types. For example, when we implement the `ArticleDocument` class, which will inherit from the `NoSQLBaseDocument` class, all the instances where `T` was used will be replaced with the `ArticleDocument` type when analyzing the signature of functions (more on Python generics: https://realpython.com/python312-typing).

The `NoSQLBaseDocument` class is then declared as an abstract base class inheriting from Pydantic's BaseModel, Python's Generic (which provides the functionality described earlier), and `ABC` (making the class abstract) classes. This class serves as the foundational ODM class:

```
T = TypeVar("T", bound="NoSQLBaseDocument")
class NoSQLBaseDocument(BaseModel, Generic[T], ABC):
```

Within the `NoSQLBaseDocument` class, an id field is defined as a UUID4, with a default factory generating a unique UUID. The class also implements the `__eq__` and `__hash__` methods to allow instances to be compared and used in hashed collections like sets or as dictionary keys based on their unique `id` attribute:

```
id: UUID4 = Field(default_factory=uuid.uuid4)
def __eq__(self, value: object) -> bool:
    if not isinstance(value, self.__class__):
        return False
    return self.id == value.id
def __hash__(self) -> int:
    return hash(self.id)
```

The class provides methods for converting between MongoDB documents and class instances. The `from_mongo()` class method transforms a dictionary retrieved from MongoDB into an instance of the class. The `to_mongo()` instance method converts the model instance into a dictionary suitable for MongoDB insertion:

```
@classmethod
def from_mongo(cls: Type[T], data: dict) -> T:
    if not data:
        raise ValueError("Data is empty.")
    id = data.pop("_id")
    return cls(**dict(data, id=id))
def to_mongo(self: T, **kwargs) -> dict:
    exclude_unset = kwargs.pop("exclude_unset", False)
    by_alias = kwargs.pop("by_alias", True)
    parsed = self.model_dump(exclude_unset=exclude_unset, by_al
    if "_id" not in parsed and "id" in parsed:
        parsed["_id"] = str(parsed.pop("id"))
    for key, value in parsed.items():
        if isinstance(value, uuid.UUID):
            parsed[key] = str(value)
    return parsed
```

The `save()` method allows an instance of the model to be inserted into a MongoDB collection. It retrieves the appropriate collection, converts the instance into a MongoDB-compatible document leveraging the `to_mongo()` method described above, and attempts to insert it into the database, handling any write errors that may occur:

```
def save(self: T, **kwargs) -> T | None:
    collection = _database[self.get_collection_name()]
    try:
        collection.insert_one(self.to_mongo(**kwargs))
        return self
    except errors.WriteError:
```

```
            logger.exception("Failed to insert document.")
            return None
```

The `get_or_create()` class method attempts to find a document in the database matching the provided filter options. If a matching document is found, it is converted into an instance of the class. If not, a new instance is created with the filter options as its initial data and saved to the database:

```
@classmethod
def get_or_create(cls: Type[T], **filter_options) -> T:
    collection = _database[cls.get_collection_name()]
    try:
        instance = collection.find_one(filter_options)
        if instance:
            return cls.from_mongo(instance)
        new_instance = cls(**filter_options)
        new_instance = new_instance.save()
        return new_instance
    except errors.OperationFailure:
        logger.exception(f"Failed to retrieve document with fil
        raise
```

The `bulk_insert()` class method allows multiple documents to be inserted into the database at once:

```
@classmethod
def bulk_insert(cls: Type[T], documents: list[T], **kwargs) ->
    collection = _database[cls.get_collection_name()]
    try:
        collection.insert_many([doc.to_mongo(**kwargs) for doc
        return True
    except (errors.WriteError, errors.BulkWriteError):
logger.error(f"Failed to insert documents of type {cls.__name__
        return False
```

The `find()` class method searches for a single document in the database that matches the given filter options:

```
@classmethod
def find(cls: Type[T], **filter_options) -> T | None:
    collection = _database[cls.get_collection_name()]
    try:
        instance = collection.find_one(filter_options)
        if instance:
            return cls.from_mongo(instance)
        return None
    except errors.OperationFailure:
        logger.error("Failed to retrieve document.")
        return None
```

Similarly, the `bulk_find()` class method retrieves multiple documents matching the filter options. It converts each retrieved MongoDB document into a model instance, collecting them into a list:

```
@classmethod
def bulk_find(cls: Type[T], **filter_options) -> list[T]:
    collection = _database[cls.get_collection_name()]
    try:
        instances = collection.find(filter_options)
        return [document for instance in instances if (document
    except errors.OperationFailure:
        logger.error("Failed to retrieve document.")
        return []
```

Finally, the `get_collection_name()` class method determines the name of the MongoDB collection associated with the class. It expects the class to have a nested `Settings` class with a name attribute specifying the collection name. If this configuration is missing, an `ImproperlyConfigured` exception

will be raised specifying that the subclass should define a nested `Settings` class:

```
@classmethod
def get_collection_name(cls: Type[T]) -> str:
    if not hasattr(cls, "Settings") or not hasattr(cls.Settings
        raise ImproperlyConfigured(
            "Document should define an Settings configuration c
        )
    return cls.Settings.name
```

We can configure each subclass using the nested `Settings` class, such as defining the collection name, or anything else specific to that subclass. Within the Python ecosystem, there is an ODM implementation on top of MongoDB, called `mongoengine`, which you can find on GitHub. It follows a pattern similar to ours but more comprehensive. We implemented it by ourselves, as it was an excellent exercise to practice writing modular and generic code following best OOP principles, which are essential for implementing production-level code.

Data categories and user document classes

The last piece of the puzzle is to see the implementation of the subclasses that inherit from the `NoSQLBaseDocument` base class. These are the concrete classes that define our data categories. You've seen these classes used across the chapter when working with articles, repositories, and posts within the crawler classes.

We begin by importing the essential Python modules and the ODM base class:

```
from abc import ABC
from typing import Optional
from pydantic import UUID4, Field
from .base import NoSQLBaseDocument
from .types import DataCategory
```

We define an `enum` class, where we centralize all our data category types. These variables will act as constants in configuring all our ODM classes throughout the book.



The class can be found in the repository at `llm_engineering/domain/types.py`.

```
from enum import StrEnum
class DataCategory(StrEnum):
    PROMPT = "prompt"
    QUERIES = "queries"
    INSTRUCT_DATASET_SAMPLES = "instruct_dataset_samples"
    INSTRUCT_DATASET = "instruct_dataset"
    PREFERENCE_DATASET_SAMPLES = "preference_dataset_samples"
    PREFERENCE_DATASET = "preference_dataset"
    POSTS = "posts"
    ARTICLES = "articles"
        REPOSITORIES = "repositories"
```

The `Document` class is introduced as an abstract base model for other documents on top of the `NoSQLBaseDocument` ODM class. It includes common attributes like content, platform, and author details, providing a standardized structure for documents that will inherit from it:

```
class Document(NoSQLBaseDocument, ABC):
    content: dict
    platform: str
```

```
    author_id: UUID4 = Field(alias="author_id")
    author_full_name: str = Field(alias="author_full_name")
```

Finally, specific document types are defined by extending the `Document` class. The `RepositoryDocument`, `PostDocument`, and `ArticleDocument` classes represent different categories of data, each with unique fields and settings that specify their respective collection names in the database:

```
class RepositoryDocument(Document):
    name: str
    link: str
    class Settings:
        name = DataCategory.REPOSITORIES
class PostDocument(Document):
    image: Optional[str] = None
    link: str | None = None
    class Settings:
        name = DataCategory.POSTS
class ArticleDocument(Document):
    link: str
    class Settings:
        name = DataCategory.ARTICLES
```

Finally, we define the `UserDocument` class, which is used to store and query all the users from the LLM Twin project:

```
class UserDocument(NoSQLBaseDocument):
    first_name: str
    last_name: str
    class Settings:
        name = "users"
    @property
    def full_name(self):
        return f"{self.first_name} {self.last_name}"
```

By implementing the `NoSQLBaseDocument` ODM class, we had to focus solely on the fields and specific functionality of each document or domain entity. All the CRUD functionality is delegated to the parent class. Also, by leveraging Pydantic to define the fields, we have out-of-the-box type validation. For example, when creating an instance of the `ArticleDocument` class, if the provided link is `None` or not a string, it will throw an error signaling that the data is invalid.

With that, we've finished implementing our data collection pipeline, starting with the ZenML components. Then, we looked into the implementation of the crawlers and, finally, wrapped it up with the ODM class and data category documents. The last step is to run the data collection pipeline and ingest raw data into the MongoDB data warehouse.

Gathering raw data into the data warehouse

ZenML orchestrates the data collection pipeline. Thus, leveraging ZenML, the data collection pipeline can be run manually, scheduled, or triggered by specific events. Here, we will show you how to run it manually, while we will discuss the other scenarios in *Chapter 11* when digging deeper into MLOps.

We configured a different pipeline run for each author. We provided a ZenML configuration file for Paul Iusztin's or Maxime Labonne's data. To call the data collection pipeline to collect Maxime's data, for example, you can run the following CLI command:

```
poetry poe run-digital-data-etl-maxime
```

That will call the pipeline with the following ZenML YAML configuration file:

```
parameters:
  user_full_name: Maxime Labonne # [First Name(s)] [Last Name]
  links:
    # Personal Blog
    - https://mlabonne.github.io/blog/posts/2024-07-29_Finetune
    - https://mlabonne.github.io/blog/posts/2024-07-15_The_Rise
    # Substack
    - https://maximelabonne.substack.com/p/uncensor-any-llm-wit
    - https://maximelabonne.substack.com/p/create-mixtures-of-e
    - https://maximelabonne.substack.com/p/merge-large-language
    … # More Substack links
```

In *Figure 3.3* earlier, we saw the pipeline's run DAG and details in ZenML's dashboard. Meanwhile, *Figure 3.5* shows the `user` output artifact generated by this data collection pipeline. You can inspect the query `user_full_name` and the retrieved `user` from the MongoDB database, for which we collected the links in this specific run.

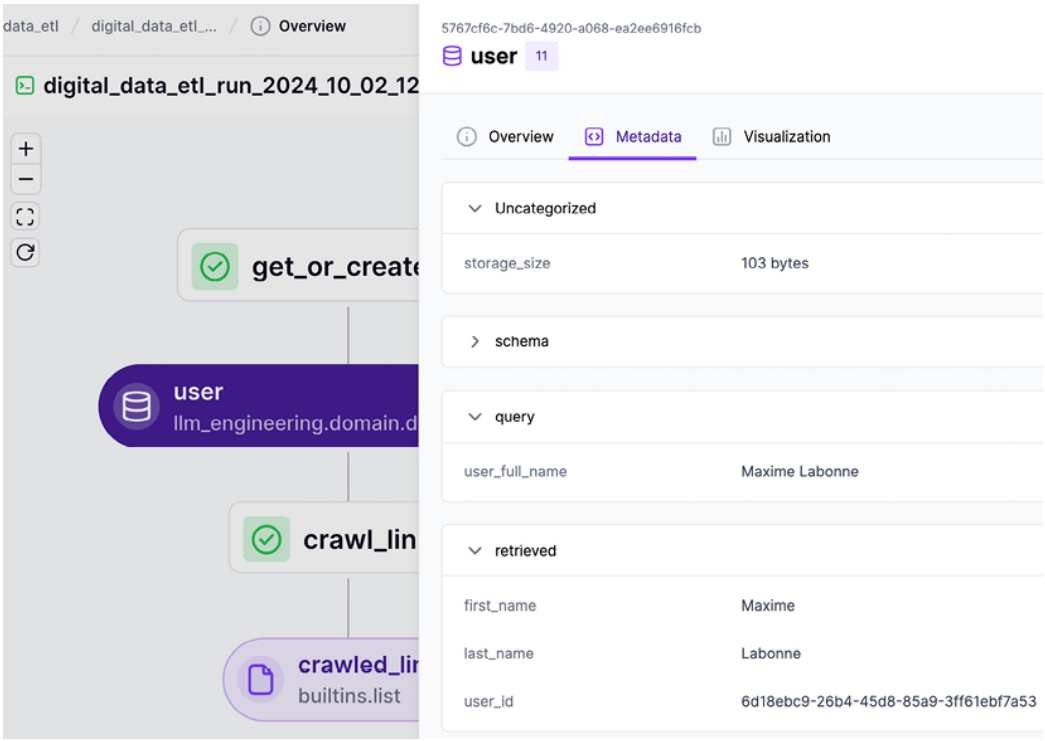


Figure 3.5: Example of the user output artifact after running the data collection pipeline using Maxime's configuration file

Also, in *Figure 3.6*, you can observe the `crawled_links` output artifact, which lists all the domains from which we collected data, the total number of links crawled for each domain, and the number of successfully collected links.

We want to highlight again the power of these artifacts, as they trace each pipeline's results and metadata, making it extremely easy to monitor and debug each pipeline run individually.

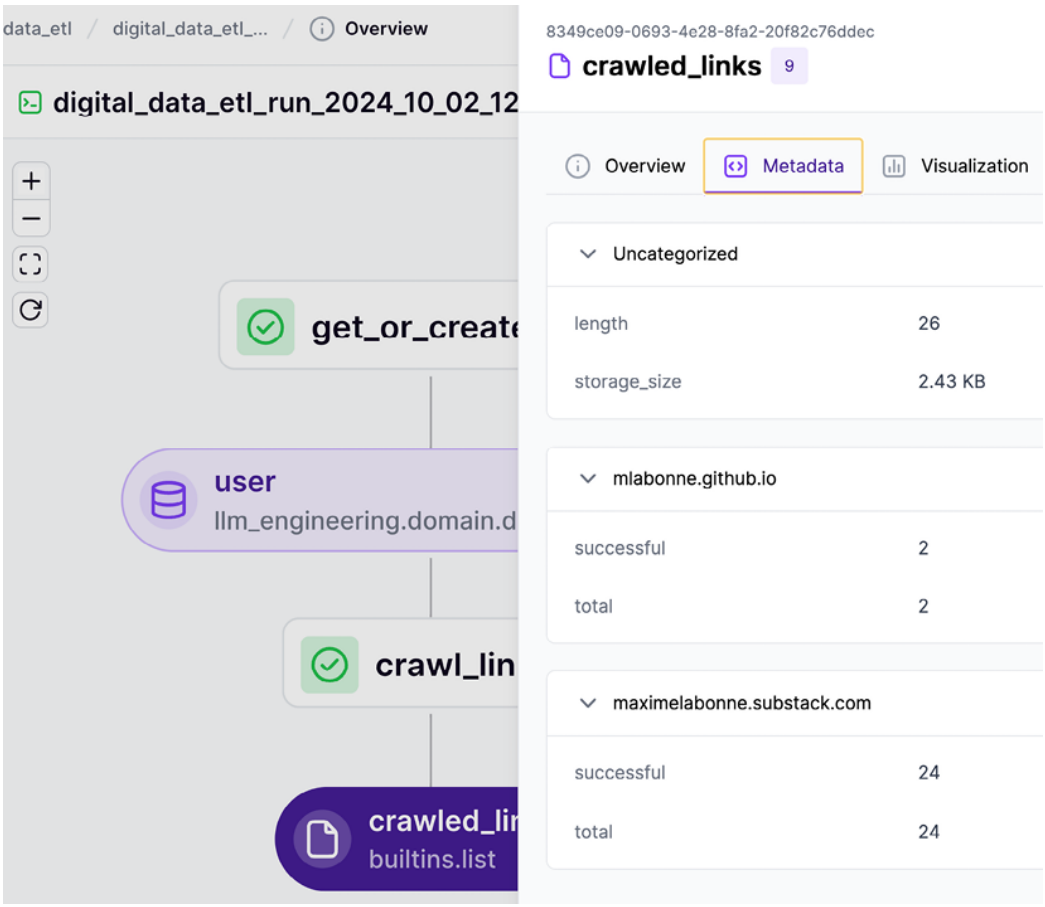


Figure 3.6: Example of the crawled_links output artifact after running the data collection pipeline using Maxime's configuration file

Now, we can download the `crawled_links` artifact anywhere in our code by running the following code, where the `ID` of the artifact can be found in ZenML and is unique for every artifact version:

```
from zenml.client import Client
artifact = Client().get_artifact_version('8349ce09-0693-4e28-8f
loaded_artifact = artifact.load()
```

For example, we can easily run the same data collection pipeline but with Paul Iusztin's YAML configuration, listed below:

```
parameters:
  user_full_name: Paul Iusztin # [First Name(s)] [Last Name]
  links:
    # Medium
    - https://medium.com/decodingml/an-end-to-end-framework-for
    - https://medium.com/decodingml/a-real-time-retrieval-syste
    - https://medium.com/decodingml/sota-python-streaming-pipel
    … # More Medium links
    # Substack
    - https://decodingml.substack.com/p/real-time-feature-pipel
    - https://decodingml.substack.com/p/building-ml-systems-the
    - https://decodingml.substack.com/p/reduce-your-pytorchs-co
    … # More Substack links
```

To run the pipeline using Paul's configuration, we call the following `poe` command:

```
poetry poe run-digital-data-etl-paul
```

That, under the hood, calls the following CLI command that references Paul's config file:

```
poetry run python -m tools.run --run-etl --no-cache --etl-config
```

You can find all the configs in the repository in the `configs/` directory. Also, using `poe`, we configured a command that calls the data collection pipeline for all the supported authors:

```
poetry poe run-digital-data-etl
```

We can easily query the MongoDB data warehouse using our ODM classes. For example, let's query all the articles collected for Paul Iusztin:

```
from llm_engineering.domain.documents import ArticleDocument, U
user = UserDocument.get_or_create(first_name="Paul", last_name=
articles = ArticleDocument.bulk_find(author_id=str(user.id))
print(f"User ID: {user.id}")
print(f"User name: {user.first_name} {user.last_name}")
print(f"Number of articles: {len(articles)}")
print("First article link:", articles[0].link)
```

The output of the code from above is:

```
User ID: 900fec95-d621-4315-84c6-52e5229e0b96
User name: Paul Iusztin
Number of articles: 50
First article link: https://medium.com/decodingml/an-end-to-end-
```

With only two lines of code, we can query and filter our MongoDB data warehouse using any ODM defined within our project.

Also, to ensure that your data collection pipeline works as expected, you can search your MongoDB collections using your **IDE's MongoDB plugin,** which you must install separately. For example, you can use this plugin for VSCode: https://www.mongodb.com/products/tools/vs-code. For other IDEs, you can use similar plugins or external NoSQL visualization tools. After connecting to the MongoDB visualization tool, you can connect to our local database using the following URI: `mongodb://llm_engineering:llm_engineering@127.0.0.1:27017`. For a cloud MongoDB cluster, you must change the URI, which we will explore in *Chapter 11*.

And just like that, you've learned how to run the data collection pipeline with different ZenML configs and how to visualize the output artifacts of each run. We also looked at how to query the data warehouse for a

particular data category and author. Thus, we’ve finalized our data engineering chapter and can move to the conclusion.

Troubleshooting

The raw data stored in the MongoDB database is central to all future steps. Thus, if you haven't successfully run the code from this chapter due to any issues with the crawlers, this section provides solutions for fixing potential issues to allow you to move forward.

Selenium issues

It is a well-known issue that running Selenium can cause problems due to issues with the browser driver, such as the `ChromeDriver`. Thus, if the crawlers that use Selenium, such as the `MediumCrawler`, fail due to problems with your `ChromeDriver`, you can easily bypass this by commenting out the Medium links added to the data collection YAML configs. To do so, go to the `configs/` directory and find all the YAML files that start with `digital_data_etl_*`, such as `digital_data_etl_maxime_labonne.yaml`. Open them and comment on all the Medium-related URLs, as illustrated in *Figure 3.7*. You can leave out the Substack or personal blog URLs as these use the `CustomArticleCrawler`, which is not dependent on Selenium.

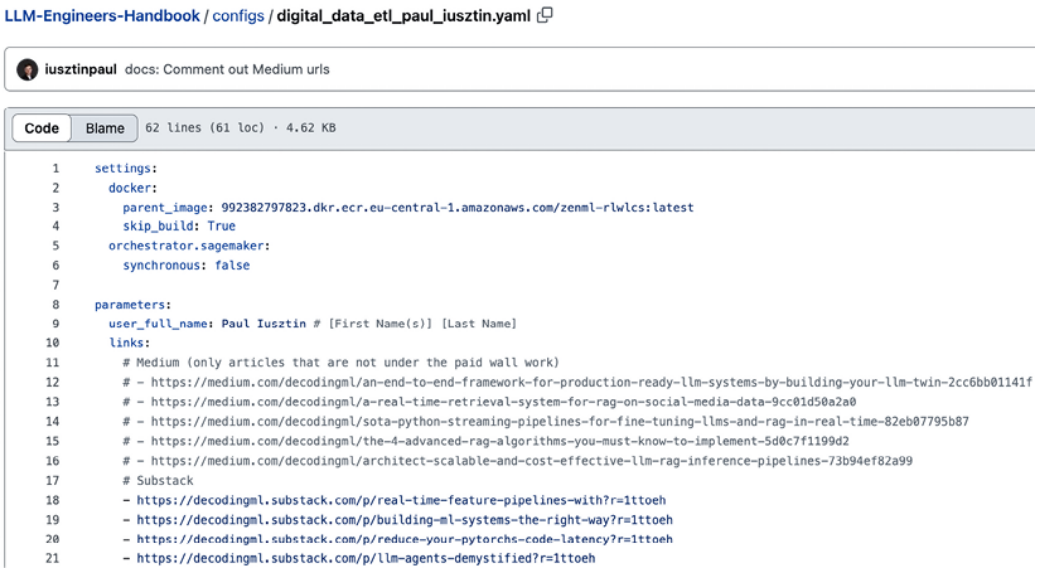


Figure 3.7: Fix Selenium issues when crawling raw data

Import our backed-up data

If nothing works, there is the possibility of populating the MongoDB database with your backed-up data saved under the `data/data_warehouse_raw_data directory`. This will allow you to proceed to the fine-tuning and inference sections without running the data collection ETL code. To import all the data within this directory, run:

```
poetry poe run-import-data-warehouse-from-json
```

After running the CLI command from above, you will have a one-to-one replica of the dataset we used while developing the code. To ensure the import is completed successfully, you should have 88 articles and 3 users in your MongoDB database.

Summary

In this chapter, we've learned how to design and build the data collection pipeline for the LLM Twin use case. Instead of relying on static datasets, we collected our custom data to mimic real-world situations, preparing us for real-world challenges in building AI systems.

First, we examined the architecture of LLM Twin's data collection pipeline, which functions as an ETL process. Next, we started digging into the pipeline implementation. We began by understanding how we can orchestrate the pipeline using ZenML. Then, we looked into the crawler implementation. We learned how to crawl data in three ways: using CLI commands in subprocesses or using utility functions from LangChain or Selenium to build custom logic that programmatically manipulates the browser. Finally, we looked into how to build our own ODM class, which we used to define our document class hierarchy, which contains entities such as articles, posts, and repositories.

At the end of the chapter, we learned how to run ZenML pipelines with different YAML configuration files and explore the results in the dashboard. We also saw how to interact with the MongoDB data warehouse through the ODM classes.

In the next chapter, we will cover the key steps of the RAG feature pipeline, including chunking and embedding documents, ingesting these documents into a vector DB, and applying pre-retrieval optimizations to improve performance. We will also set up the necessary infrastructure programmatically using Pulumi and conclude by deploying the RAG ingestion pipeline to AWS.

References

- Breuss, M. (2023, July 26). *Beautiful Soup: Build a Web Scraper With Python*. https://realpython.com/beautiful-soup-web-scraper-python/
- David, D. (2024, July 8). *Guide to Web Scraping with Selenium in 2024*. Bright Data. https://brightdata.com/blog/how-tos/using-selenium-for-web-scraping
- Hjelle, G. A. (2023, October 21). *Python 3.12 Preview: Static Typing Improvements*. https://realpython.com/python312-typing/
- *ORM Quick Start — SQLAlchemy 2.0 documentation*. (n.d.). https://docs.sqlalchemy.org/en/20/orm/quickstart.html
- Ramos, L. P. (2023, August 4). *Python and MongoDB: Connecting to NoSQL Databases*. https://realpython.com/introduction-to-mongodb-and-python/
- Refactoring.Guru. (2024, January 1). *Builder*. https://refactoring.guru/design-patterns/builder
- *What is ETL? A complete guide*. (n.d.). Qlik. https://www.qlik.com/us/etl

Join our book’s Discord space

Join our community’s Discord space for discussions with the authors and other readers:

https://packt.link/llmeng

